KRPCLAB · BOOSTERLAND

How the booster lander works

Bringing a spent first stage back to the KSC launch pad, Super Heavy style, from a kRPC script

> The booster separates at about 20 km, still moving downrange at several hundred metres per second. The autopilot flips it, burns back toward the pad, falls through the atmosphere using the body as a weak wing, and lights the engines once more for a suicide burn that puts the legs on a pad about 20 m across.

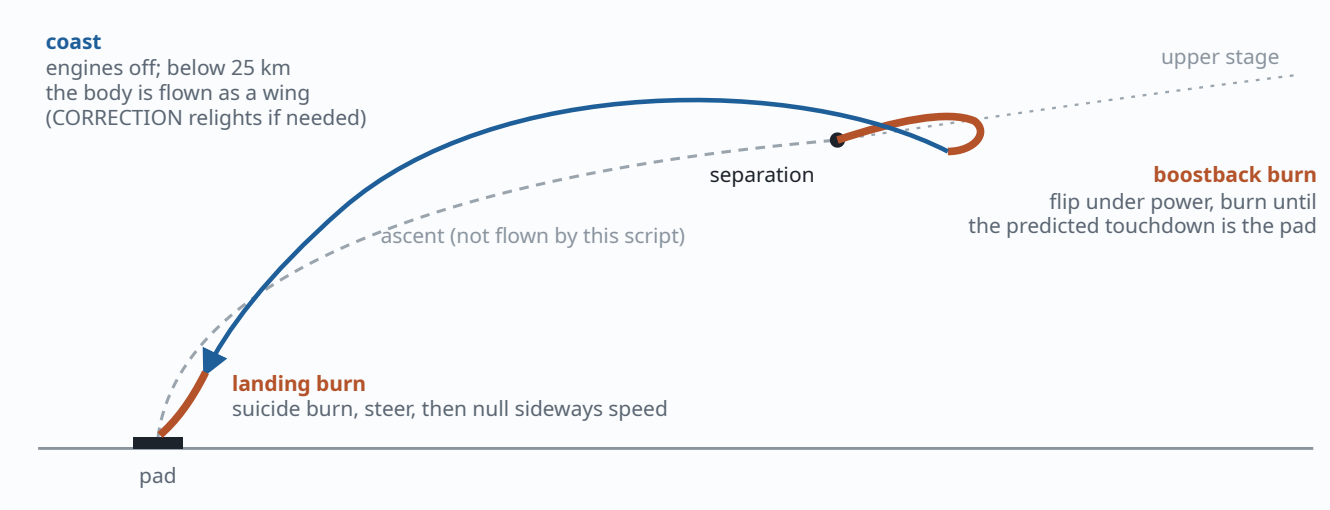


Figure 1. The flight profile, not to scale. Orange is engine-on; blue is unpowered. Every part of it is re-decided each tick from a fresh prediction of where the booster will touch down.

The one idea: re-predict the landing every tick

There is no stored plan. About twenty times a second the autopilot reads the booster's position and velocity, then **integrates the rest of the flight to the ground** in local Python: gravity, drag, body lift, the planet's rotation, and the landing burn itself. Whatever it decides that tick, it decides from that fresh prediction: which way to point, how much throttle, when boostback is done, when to light the landing burn, which way to steer. A disturbance that sends the booster off course shows up in the next prediction, so it gets corrected without anything having to detect it as a disturbance.

The code is organised around that idea. `autoland.py` is the phase machine and the only code that talks to the game. `trajectory.py` is the propagator. `guidance.py` holds the control laws as pure functions that can be tested without KSP. `environment.py` keeps the world data the propagator needs: air density, speed of sound, a drag curve and a lift slope. It caches all of it so the integration loop makes no remote calls at all.

What the predictor has to get right

- **It flies the landing burn too.** The prediction coasts to where the suicide burn will start, then keeps integrating with the engines on, using the same throttle law the vehicle will fly (`trajectory.landing_command` is shared by both). Braking sheds horizontal speed, so the real touchdown is short of the ballistic impact point, by kilometres for a fast booster. A boostback that aimed at the ballistic point would overshoot every time.

- **RK4.** Drag goes as v^2 through a density that changes by a factor of e every 5 km, so the integration is fourth-order: a first-order step over the descent would bias the predicted touchdown, not just add noise.
- **Drag is a curve against Mach.** Cd·A is probed with kRPC's `simulate_aerodynamic_force_at`, which takes the velocity to evaluate as an argument, so the whole curve is measured from one position. It is re-swept once the booster is falling nose-first.
- **The planet's rotation is measured.** The rotation vector is found by comparing the vessel's velocity in the rotating and inertial frames (kRPC frames are left-handed).
- **Heights are measured from the feet.** The trigger height uses the terrain under the legs, clamped against the pad's own radius.

The flight, phase by phase

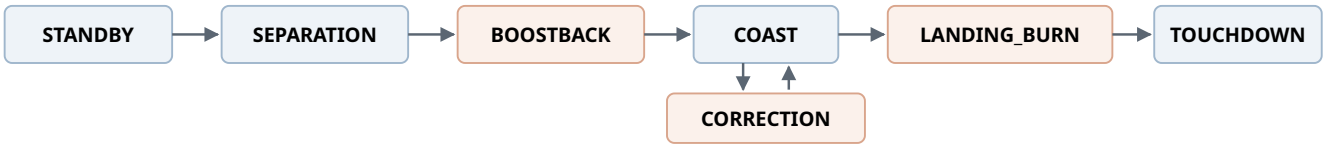


Figure 2. The phase machine. Orange phases burn the engines. CORRECTION is a short relight that COAST can enter and return from.

STANDBY
waits for START

On START: RCS on, every airbrake deployed at 90° with pitch and yaw control, every liquid engine activated, and the autopilot engaged on the current attitude.

SEPARATION
a few seconds

Decides *when it is safe to flip*. `proximity.py` wraps each nearby craft in a sphere and marches the booster's motion forward against each one, both coasting and at full thrust along the boostback aim. The phase ends on the first tick where nothing is in the way.

The two stages start overlapping by construction, so the keep-out distance is capped at the gap they are already sitting at: only getting *closer* counts as a conflict. A 5 s backstop flips anyway.

BOOSTBACK
flip and burn

Aim opposite the predicted horizontal miss and go to **full throttle on the first tick, before the turn is finished**, so the engine gimbal (the strongest attitude actuator, present only while burning) does the turning. Big turns are routed *over the top*, so the engines never swing down into the airstream.

Throttle tapers off as the remaining Δv falls, and the burn ends when the predicted touchdown, landing burn included, is within 190 m of the pad. Roll is commanded as well as pointing, so the grid fins sit in the plane they steer in.

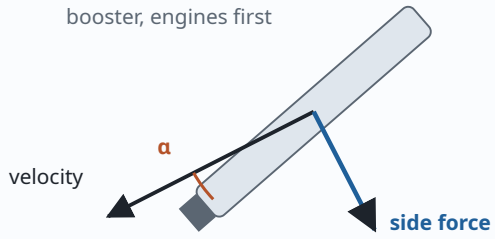
COAST
engines off

Fall back toward the pad, watching for the moment the landing burn has to start (the predicted ignition point). Above 25 km it holds retrograde. Below 25 km it **flies the body as a wing** (next section). If the predicted miss grows past 60 m → CORRECTION; at the ignition point → LANDING_BURN.

CORRECTION relight, if needed	If the predicted miss grows past 60 m while there is still height to use, a throttled-down relight closes it to under 15 m and returns to COAST. A falling booster can't hold boostback's horizontal attitude in thick air, so it steers within 50° of retrograde. Before each burn it *propagates twice* to measure how many metres of miss one m/s actually closes, and it never fires a burn that would make the miss worse. If the booster is stuck off-aim and not turning, it lights at 25% throttle so the gimbal can turn it.
LANDING_BURN suicide burn	Gear down on the first tick. Throttle follows a constant-deceleration profile (feed-forward plus a proportional trim) that blends into a constant-rate final descent, so the legs arrive at 2 m/s. The burn also steers, with ZEM/ZEV terminal guidance aimed at the propagator's predicted touchdown. Below 250 m it **stops aiming and only nulls sideways velocity** (see “Aim, then stop”).
TOUCHDOWN hand back	Throttle zero, log the distance from the pad, exit. Every exit path (TERMINATE, Ctrl-C, a crash) runs the same shutdown: throttle zero, autopilot off, SAS on, panel removed, connection closed.

Flying the coast as a wing

Engine burns can only *shorten* the flight: CORRECTION steers near retrograde and the landing burn thrusts against the velocity. The air can push in **both** directions. Below 25 km the booster is held a few degrees off retrograde, and the angle is solved each tick to null the predicted miss.



Solve, then verify

1. Measure the side-force slope with the same probe that measures drag (no lift found = hold retrograde).
2. Propagate small test angles in-plane and out-of-plane; invert the 2×2 to null downrange and cross-track.
3. Propagate the answer. Keep it only if it lands nearer than doing nothing (else halve it once, then give up).

Figure 3. Tilting the booster a few degrees off its velocity turns the airflow into a sideways force. The force goes roughly as $\alpha \cdot |\alpha|$ (a cylinder in crossflow, not a wing), so both the propagator and the solver are quadratic in the angle.

The side-force slope is *measured* each flight with the same probe as the drag; a vehicle with none is not steered. On this booster the lift is body lift (grid fins change drag, not side force), L/D about 0.04, which gives roughly 300 m of authority over the last 40 s.

TWO GUARDS

Start at 25 km. Above that the prediction is still settling while the drag curve is re-measured nose-first, so the wing waits rather than steer against a moving estimate.

Check the answer before flying it. The commanded angle is re-propagated and kept only if it lands nearer than doing nothing (else halved once, else dropped). Near the ground the sensitivity collapses and a raw solve would return the maximum angle.

Aim, then stop

The landing burn steers with ZEM/ZEV (zero-effort-miss / zero-effort-velocity) guidance, gains 6 and 4 from the energy-optimal solution, with the tilt capped so steering never takes thrust from braking. Throttle follows a constant-deceleration profile that blends into a constant-rate final descent, so the legs arrive at 2 m/s.

Below 250 m the position term is dropped and the law runs on its velocity term alone, down to 4 m: the last stretch only nulls sideways speed, so the legs arrive vertical.

Smaller rules

- **Burn for the gimbal.** If CORRECTION is far off its aim *and not turning toward it* (under 3 °/s), it lights at 25% throttle so the gimbal turns it.
- **RCS is a relay.** Thrusters open when the pointing error is large and close once it has *stayed* small for a moment.
- **Roll is held.** The roof is rolled into the trajectory plane so the grid fins steer the axis that matters; the roll reference is seeded from the zenith once and carried forward.
- **The landing burn does not divert.** Boostback, the coast and CORRECTION remove the miss; the landing burn stops the vehicle.

How it is measured

Each flight writes a numbered log (`logs/LOG<n>`). Telemetry is paced in *game* time, so a paused game logs nothing and time warp still gives one line per interval. Every log records which settings differ from the defaults *and* a fingerprint of the defaults themselves, so a flight can always be filed under the right configuration. Tunables live only in `config.py`; `--set FIELD=VALUE` overrides any of them for one flight.

Five reference saves (`quicksave`, `qs_hot`, `qs_cold`, `qs_steep`, `qs_north`) give five separation states, and `tools/savegen.py` writes more. `tests/fakeksp` runs the closed loop offline for screening.

State of the code on 2026-10-09. Tunables: `boosterland/config.py`.